THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF SCIENCE

Assignment 1 Report: A Policy-Gradient-Based Study with Constraint-Aware Portfolio Learning

Student:
LI, Jiaxuan

Supervisor:
Prof. Wong, Chak

Summary

This report studies **reinforcement learning for dynamic asset allocation under realistic portfolio constraints**. The core task is to learn a mapping from observable market states to portfolio weights that maximizes **long-horizon investor welfare**, rather than one-step expected return. The project builds a modular framework with a portfolio environment, controlled experimental settings, and three agent classes: **policy gradient (REINFORCE)**, **linear actor-critic**, and **neural actor-critic**.

The report is organized around a clear logic. First, it restates dynamic asset allocation as a constrained sequential decision problem. Second, it explains why **policy gradient** is the conceptual starting point, and how the model was progressively improved through **variance reduction, critic-based learning, nonlinear function approximation, simplex projection, and turnover-aware execution**. Third, it presents an experimental design aimed not only at measuring performance, but at verifying whether training is actually **stable and convergent**. Finally, it interprets the learned behavior economically, emphasizing that portfolio constraints—especially turnover limits—can function not only as frictions, but also as **regularizers that improve learning quality**.

A central conclusion of the project is that reinforcement learning can be a credible framework for constrained portfolio choice only when **optimization, feasibility, and validation** are designed together. In particular, the project shows that convergence should not be judged by reward alone, but by a combination of training stability, policy behavior, conditional allocation patterns, and robustness across controlled environments.

1. Introduction

Asset allocation is inherently a **dynamic decision problem**. At each date, an investor observes market information, adjusts portfolio weights, receives realized returns, and continues over time. The quality of any decision therefore depends not only on immediate payoff, but also on how that decision affects future wealth evolution, trading flexibility, and exposure to risk.

This sequential structure makes reinforcement learning a natural methodological candidate. Unlike static portfolio optimization, reinforcement learning can directly learn a **state-dependent allocation rule** through repeated interaction with an environment. However, portfolio learning differs from standard reinforcement learning problems in several important ways:

- actions are **continuous** rather than discrete;
- allocations must satisfy **financial feasibility constraints**;
- portfolio changes are often limited by **turnover or trading frictions**;
- returns are **noisy and high-variance**, making optimization unstable;
- success must be evaluated not only numerically, but also **economically**.

Because of these issues, this project is not simply an application of an RL algorithm to a financial simulator. Its main objective is to build a framework in which the learned policy is both **trainable** and **economically interpretable**. The report therefore focuses on three questions:

1. How should dynamic asset allocation be formulated as a constrained RL problem?
2. Why is policy gradient the right conceptual starting point, and how was it improved in practice?
3. How can we verify that the learned policy is actually converging in a meaningful sense?

2. Problem Restatement

Got it. Here is the concise, professional version of Section 2 in English, with standardized LaTeX formulas for your report.

2.1 Dynamic Asset Allocation as Sequential Control

The portfolio problem is modeled as a **Markov Decision Process (MDP)**. At each time step t, the agent observes a market stateS_t, selects a portfolio action a_t, receives a reward r_t, and transitions to the next state s_(t+1). A trajectory is denoted by:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$$

The policy is parameterized by θ and written as π_θ. The objective is to maximize the expected cumulative utility:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

This formulation acknowledges that asset allocation is not a one-step prediction problem. A decision today impacts future wealth evolution, rebalancing flexibility, and the long-term path of compounding.

2.2 Portfolio-Specific Constraints

Unlike generic continuous control, the action a_t represents a vector of portfolio weights w_i, which must satisfy **Long-only Simplex Constraints**:

$$w_i \geq 0, \quad \sum_{i=1}^{N} w_i = 1$$

To ensure realism, the project enforces a **Turnover Constraint**. The policy's output is transformed through two feasibility mechanisms before execution:

1. **Simplex Projection:** To ensure non-negative weights that sum to unity.
2. **Turnover-Limited Adjustment:** To ensure the transition from current holdings to the target stays within a realistic trading budget.

2.3 Utility-Based Objective

The project evaluates policies using **CARA utility** and terminal wealth statistics rather than raw returns. This aligns with the financial principle that investors seek to maximize **risk-adjusted welfare**. A policy is successful only if it produces stable, interpretable wealth outcomes under realistic trading restrictions.

3. Model Choice and Model Evolution Path

This section explains the core methodological logic of the project: **why policy gradient was chosen**, and **how it was progressively improved to address the difficulties of financial RL**.

This program did not use value iteration **because the portfolio allocation problem in this project has a continuous action space**: the action is a vector of portfolio weights rather than a small set of discrete choices. Classical value iteration is most suitable when the state and action spaces are discrete and relatively small, since it requires evaluating and maximizing the value over all possible actions at each step. In my setting, portfolio weights must also satisfy simplex constraints and turnover limits, which makes exact value iteration impractical. By contrast, policy gradient methods can directly parameterize and optimize a continuous allocation policy, making them more natural and scalable for dynamic asset allocation.

3.1 Why policy gradient is the conceptual starting point

Policy gradient was chosen as the foundation for three reasons.

- Direct optimization of the allocation rules
- Natural support for continuous actions.
- Alignment with long-horizon utility objectives

Portfolio performance depends on wealth compounding, future path effects, and nonlinear utility. Policy gradient can optimize trajectory-level objectives without needing to reduce the problem to one-step prediction or analytically simplified control rules.

The REINFORCE update takes the form

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]$$

where G_t is return-to-go. Its interpretation is transparent: actions associated with better long-run outcomes should become more likely.

3.2 Baseline model: REINFORCE

The first implemented model is a **policy-gradient agent** using a linear Gaussian policy. Training proceeds by:

1. sampling full trajectories from the current policy;
2. computing return-to-go;
3. updating policy parameters according to REINFORCE.

This model is the cleanest theoretical starting point because it directly reflects the policy-gradient principle with minimal additional machinery.

However, in practice, pure REINFORCE is unstable in financial environments because rewards are noisy and trajectory returns have high variance. This is the main reason the project evolved beyond the baseline.

3.3 First optimization: variance reduction with baselines

The first improvement was to add **baseline subtraction**:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (G_t - b(s_t))\right]$$

This does not change the expected gradient direction, but it reduces variance substantially. The project implements three baseline configurations:

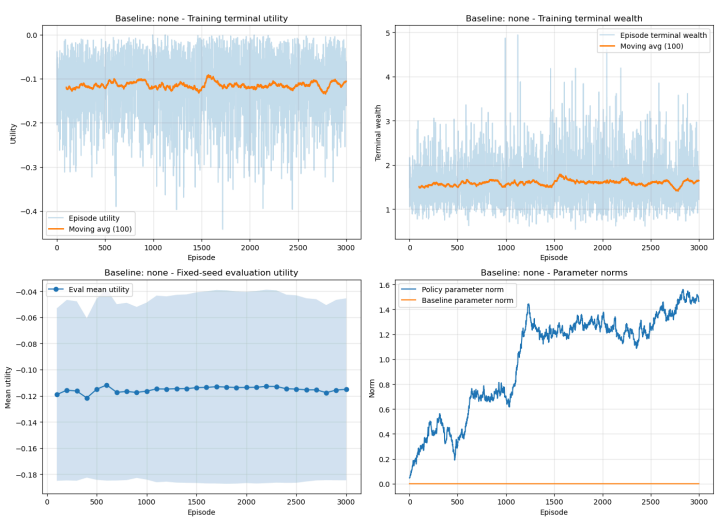
- no baseline;
- linear baseline;
- neural baseline.

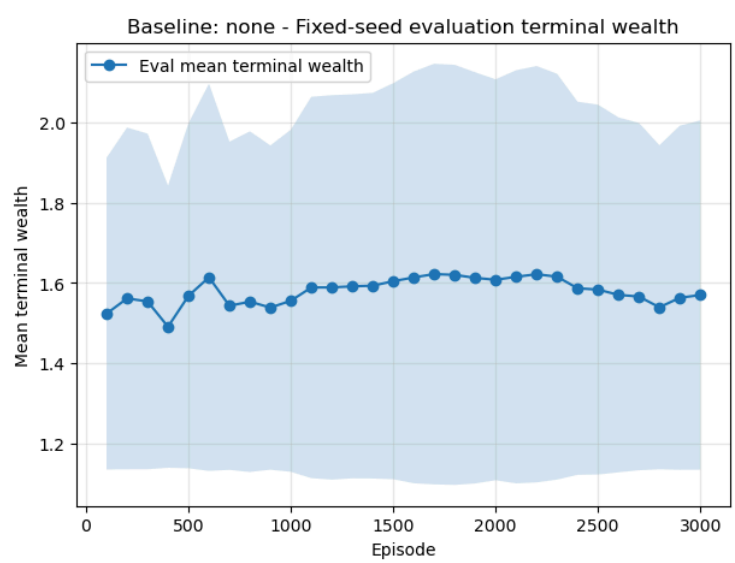
The role of this step is purely optimization-oriented: it makes policy-gradient learning less noisy and more stable, especially when rewards are path-dependent and utility-sensitive.

3.3.1 Model Results

No Baseline

For the **no-baseline** setting, training utility and terminal wealth are both highly noisy, although their moving averages remain relatively stable over time. In fixed-seed evaluation, the mean utility stays around −0.12and the mean terminal wealth is roughly 1.5-1.6, indicating modest but fairly stable performance. The policy parameter norm keeps growing throughout training, which suggests the policy is learning, but the absence of a baseline does not reduce gradient variance effectively.

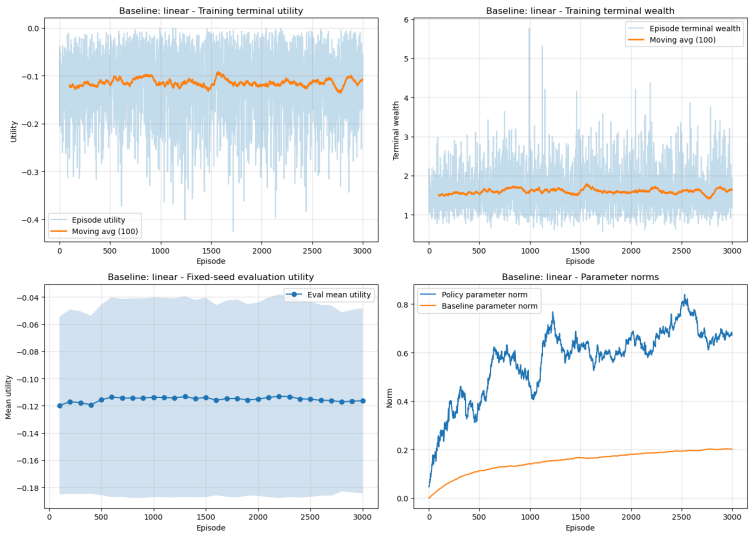


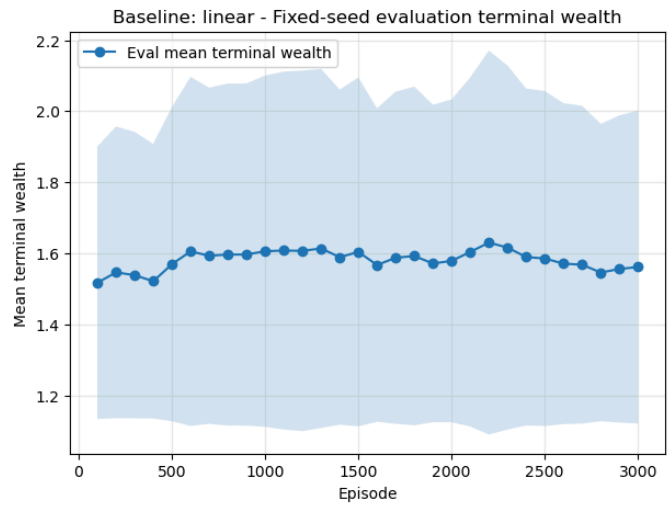


Linear Baseline

For the **linear baseline**, the overall behavior is very similar to the no-baseline case, but the learning dynamics appear slightly more stable. The training curves still show substantial episode-level variance, yet the moving averages of utility and terminal wealth are smooth and

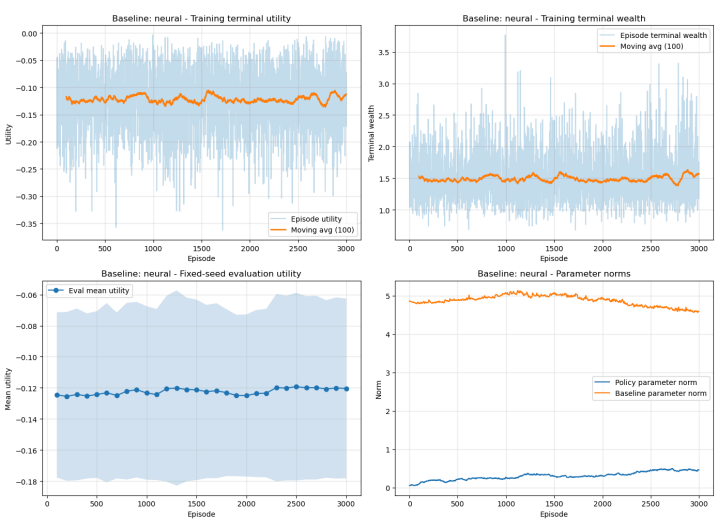
consistent. In evaluation, the linear baseline achieves slightly better terminal wealth than the other two settings, while utility remains at a comparable level. Its parameter norms also evolve smoothly, suggesting that the linear baseline provides some variance reduction without adding much complexity.

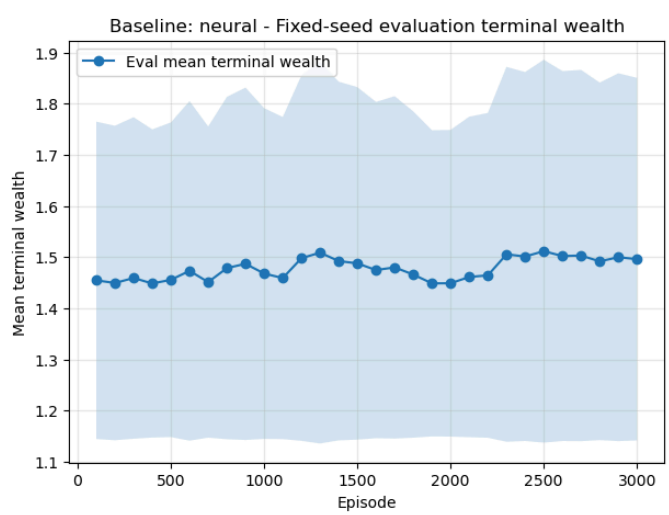




Neural Baseline

For the **neural baseline**, training and evaluation performance remain broadly comparable to the other two methods, with mean utility still close to −0.12and mean terminal wealth around 1.5. However, the neural baseline does not deliver a clear improvement despite being more expressive. The parameter norm plot shows that the baseline network carries much larger parameter magnitude than the policy, implying that much of the learning capacity is absorbed by the baseline model. Overall, this added complexity does not seem to translate into better portfolio performance in this experiment.





3.4 Second optimization: actor-critic as a lower-variance policy-gradient method

The second improvement was to move from Monte Carlo policy gradient to **actor-critic**. In this setup:

- the **actor** remains the policy,
- the **critic** estimates state value,
- policy updates use an advantage-like signal instead of raw return-to-go.

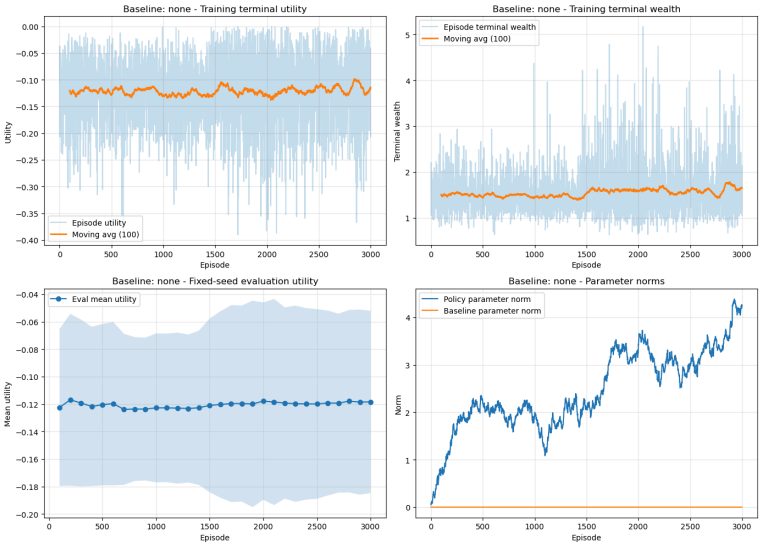
This reduces variance further because the policy no longer depends entirely on noisy full-trajectory outcomes. To be specific, I replaced Monte Carlo training with temporal-difference (TD) learning because Monte Carlo methods estimate returns using complete sampled trajectories, which often leads to high-variance gradient or value estimates. In portfolio allocation, where returns are stochastic and market noise is large, this variance can make optimization slow and unstable. TD learning addresses this issue by bootstrapping from the estimated value of the next state, so updates can be performed at every step rather than only at the end of an episode. As a result, TD learning is generally more sample efficient, provides faster credit assignment, and can significantly improve training stability, even though it may introduce some bia

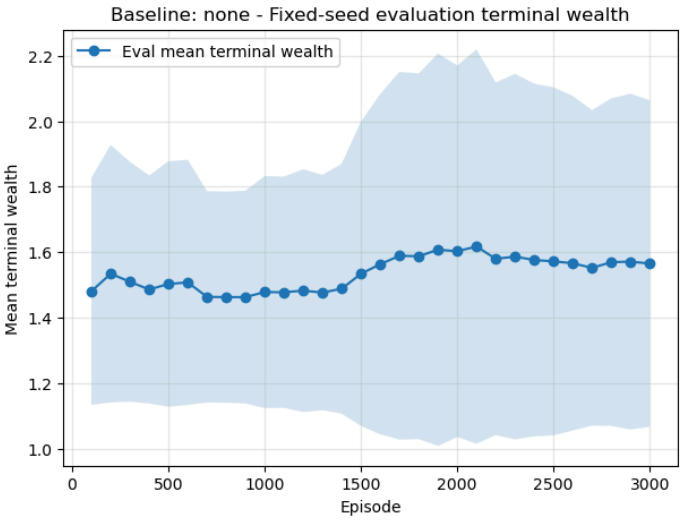
This model should be understood not as a different idea from policy gradient, but as a more efficient implementation of the same objective.

3.4.1 Model Results

No Baseline

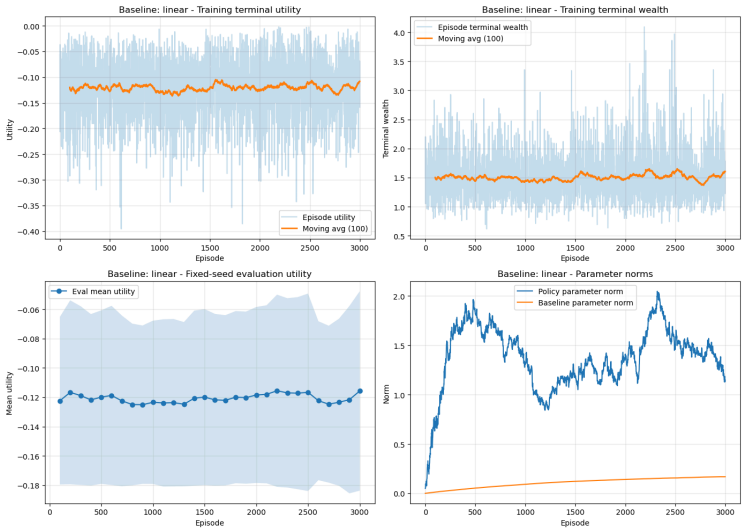
Under TD learning with no baseline, the training curves remain noisy at the episode level, but the moving averages are relatively stable and show a mild upward trend in terminal wealth over time. In fixed-seed evaluation, the mean utility stays close to −0.12, while mean terminal wealth improves from roughly 1.48 to around 1.60 before stabilizing slightly lower. This suggests that TD learning is able to produce gradual policy improvement even without an explicit baseline, although the policy parameter norm grows quite strongly, indicating a less controlled optimization process.

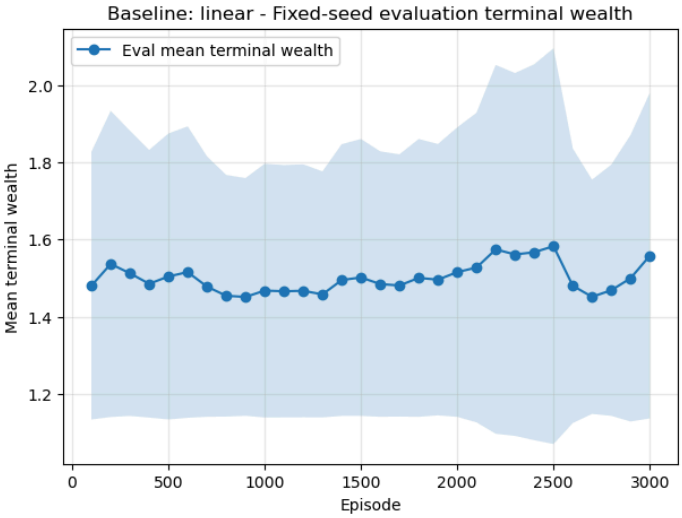




Linear Baseline

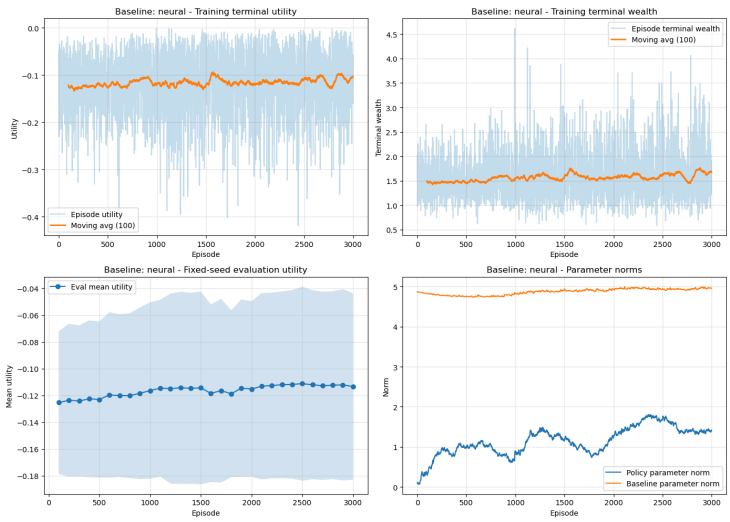
With the linear baseline, TD learning produces similarly noisy training episodes, but the smoothed curves look stable and consistent throughout training. The evaluation utility remains around −0.12, while the evaluation terminal wealth shows a moderate improvement and reaches about 1.56-1.58, near the later stage of training. Compared with the no-baseline case, the parameter dynamics are more moderate: the policy norm fluctuates within a smaller range and the baseline norm grows smoothly. This indicates that the linear baseline helps stabilize TD updates without changing final performance dramatically.

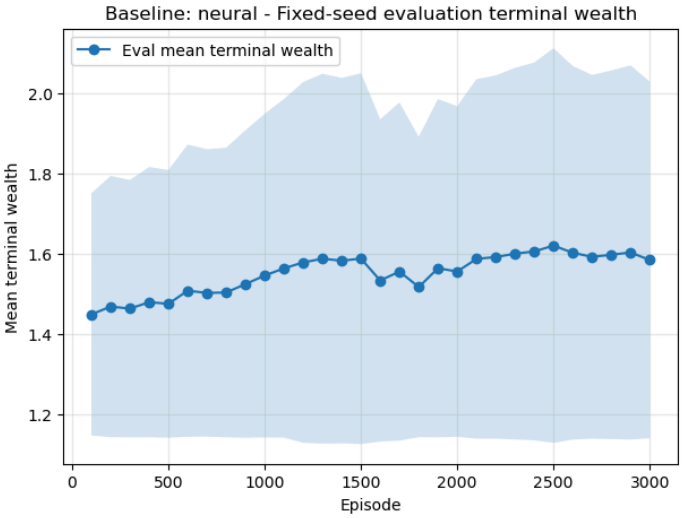




Neural Baseline

For the neural baseline, TD learning gives the strongest overall evaluation performance among the three settings. Although the training utility and wealth are still noisy, their moving averages trend slightly upward, and the fixed-seed evaluation results are the best: the mean utility improves to around −0.11 to −0.113, and the mean terminal wealth rises from about 1.45 to approximately 1.60 -- 1.62. The parameter norm plot shows that the neural baseline has large and stable parameters, while the policy norm evolves more moderately. This suggests that under TD learning, the neural baseline provides more effective value estimation and better variance reduction, leading to the most consistent evaluation gains.





3.5 Third optimization: neural actor-critic

The final model introduces **nonlinear function approximation**:

- a neural Gaussian actor,
- a neural value critic.

This is motivated by the fact that state-dependent portfolio rules may not be well represented by linear mappings, especially in the regime-switching environments. Neural approximation allows richer state-action relationships and improves expressiveness.

Conceptually, this is still a policy gradient. The policy is still the central object being optimized. The difference is that both policy representation and value estimation are now more flexible.

3.5.1 Model Results

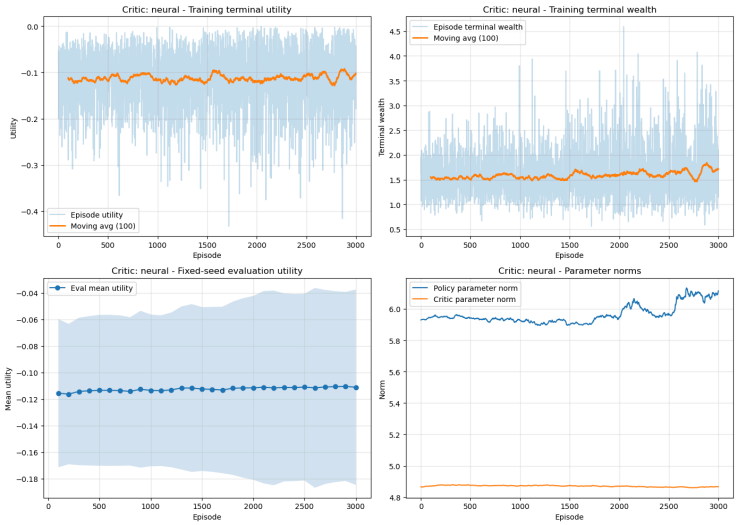
Neural Guassian Gradient Policy

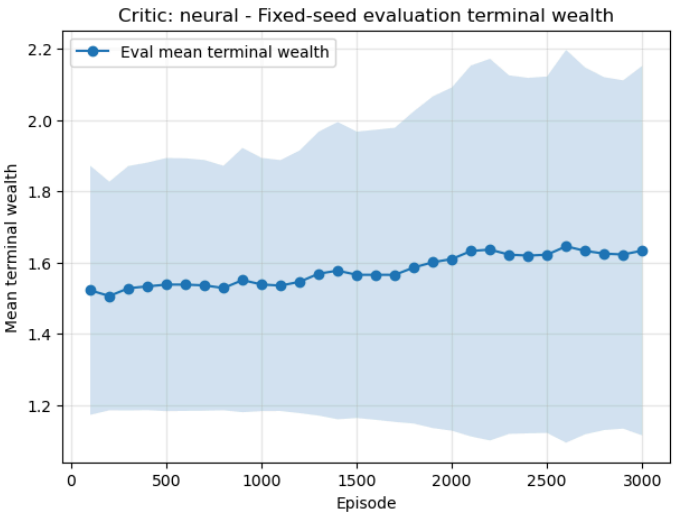
For the neural policy with a neural critic, the training curves remain noisy at the episode level, which is expected in a stochastic portfolio environment. However, the 100-episode moving averages show a clearer improving trend than in the simpler settings. In particular, the moving average of terminal utility gradually improves from about −0.12 to around −0.10, while the moving average of terminal wealth increases from roughly 1.5 to about 1.7. This suggests that the neural policy is able to learn a better allocation rule over time, even though individual episodes still exhibit high variance.

The fixed-seed evaluation results confirm this improvement. The evaluation mean utility becomes slightly less negative during training, rising from around −0.116 to about −0.111. At the same time, the evaluation mean terminal wealth improves more visibly, increasing from approximately 1.52 to about 1.63-1.64. Since these evaluation curves are computed on a fixed set of seeds, they indicate genuine policy improvement rather than random fluctuations in training episodes. Compared with the earlier baselines, this neural actor-critic setting appears to achieve the strongest evaluation wealth and one of the best utility levels.

The parameter norm plot also suggests stable optimization. The policy parameter norm stays around 5.9-6.1 and increases only gradually, while the critic parameter norm remains very stable near 4.87. This means the model is not showing signs of parameter explosion or severe instability. Instead, both actor and critic seem to evolve in a controlled manner, which is consistent with the relatively smooth evaluation improvement.

Overall, these results suggest that using a neural policy together with a neural critic improves representational power and allows the agent to learn more effective portfolio decisions under TD training. While the training trajectories are still noisy, the evaluation curves show that the learned policy generalizes better and achieves higher terminal wealth. This indicates that the additional flexibility of the neural policy is beneficial in this portfolio optimization problem.





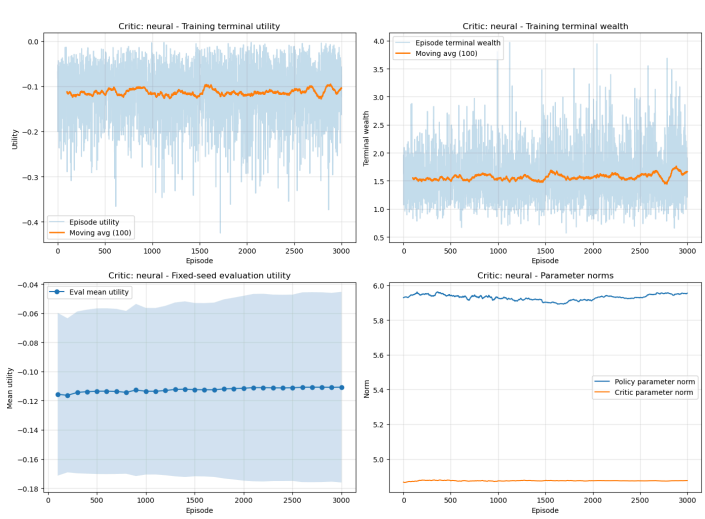
3.6 Neural Policy with Variable Learning Rate Optimizer

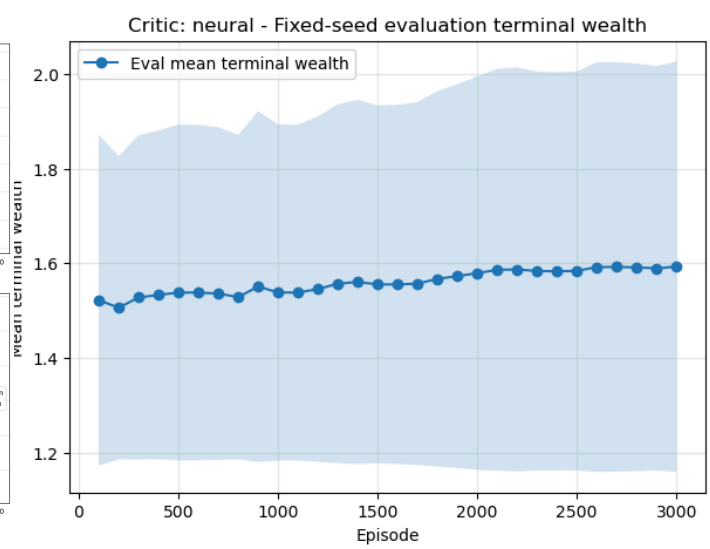
In the previous neural policy setting, both the actor and critic were trained with fixed learning rates. To improve training stability, a variable learning rate optimizer was introduced in this section. The main idea is that larger learning rates are useful in the early stage of training, when the model needs to learn quickly from noisy trajectories, while smaller learning rates are preferable in the later stage, when the policy and value function are closer to convergence. Therefore, StepLR schedulers were added to both the actor and critic optimizers, so that the learning rates decay automatically during training. In the implementation, the actor learning rate was initialized at 3×10−43 \times 10^{-4}3×10−4 and the critic learning rate at 1×10−41 \times 10^{-4}1×10−4, and both were reduced by a factor of 0.50.50.5 every 1000 episodes.

This modification does not change the neural policy architecture or the reinforcement learning framework itself. The policy still outputs Gaussian logits, which are transformed into portfolio weights through the softmax function, and the critic still estimates the value function for temporal-difference updates. The only difference is that the optimization process becomes more adaptive over time. This design is expected to make the early training phase more efficient and the later training phase more stable, thereby helping the model achieve smoother convergence in utility and terminal wealth.

3.6.1 Model Results

The variable learning rate optimizer makes the neural actor-critic training more stable and leads to a small but consistent improvement in evaluation performance. However, the improvement is moderate rather than dramatic, suggesting that the scheduler mainly helps smooth convergence instead of fundamentally changing the policy quality.





3.7 Feasibility-aware optimization

Beyond variance reduction and critic design, the project also made two critical optimization choices tied directly to portfolio realism:

Simplex projection

Raw policy outputs are projected onto the long-only simplex so that all actions correspond to valid portfolios.

Turnover-aware execution

The environment does not simply accept the policy's target weights. It applies a turnover-limited movement from the current allocation toward the target. This means training occurs under the same frictions that would constrain implementation.

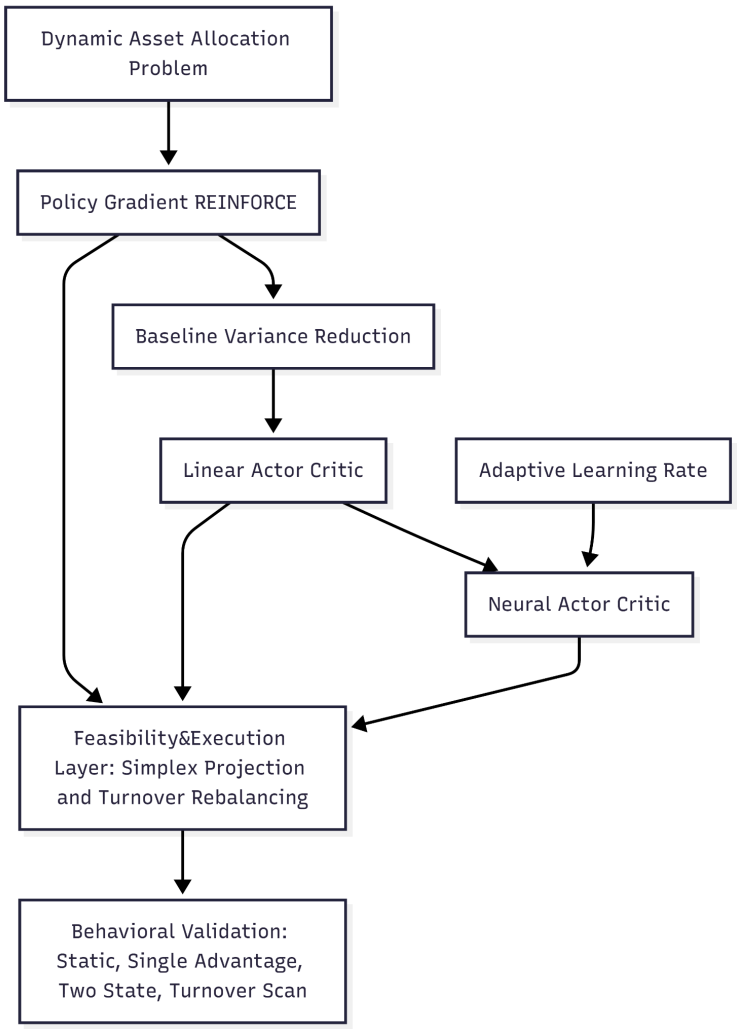
These choices improve more than realism. They also improve optimization quality by preventing the policy from exploiting infeasible or excessively unstable actions.

3.8 Model evolution path: summary

The logic of model development can be summarized as follows:

1. **Policy gradient** provides the correct conceptual foundation.
2. **Baselines** reduce Monte Carlo variance.
3. **Actor-critic** improves sample efficiency and training stability.
4. **Neural function approximation** enables richer state-dependent allocation rules.
5. **Simplex projection and turnover control** ensure both feasibility and regularized learning behavior.

So the project is best understood not as three disconnected models, but as a **progressive refinement of a policy-gradient portfolio learner**.



4. Convergence Validation: Experimental Design and Results

This section should emphasize that the project does not treat convergence as "training reward went up once." Instead, convergence is validated through **controlled environments, repeated diagnostics, and behavioral consistency checks**.

4.1 Convergence Definition

In a financial RL setting, convergence should be interpreted at three levels:

Optimization convergence

Whether the learning process becomes stable over training, with reduced volatility in updates and no persistent numerical divergence.

Policy convergence

Whether the learned allocation rule becomes stable and repeatable under the same environment.

Economic convergence

Whether the stabilized policy exhibits economically sensible behavior, rather than simply exploiting randomness.

Therefore, convergence is verified not by one metric, but by a collection of indicators:

- training utility trajectory,
- evaluation utility trajectory,
- final wealth distribution,
- critic loss behavior,
- average turnover,
- conditional state accuracy,
- regime-specific allocation response.

4.2 Experiment 1: Static benchmark

Design purpose

The static benchmark is used to verify basic training correctness. It checks whether the model can learn a feasible and nondegenerate policy in a stationary market.

What convergence means here

In this environment, convergence is indicated by:

- stable training and evaluation utility,
- no pathological oscillation in allocations,
- consistent wealth evolution,
- reasonable turnover behavior.

Why it matters

If the algorithm cannot stabilize in a stationary benchmark, then convergence claims in more complex environments are not credible.

Results & Interpretation

The same as the parameter stability which is stated in section3.

4.4 Experiment 2: Single-advantage environment

Design purpose

This environment introduces a persistent cross-sectional signal: one asset is structurally better than the others. It tests whether the learned policy converges to a behavior that recognizes and exploits a stable advantage.

What convergence means here

Convergence is validated if:

- the policy consistently overweights the advantaged asset,
- this overweighting persists across evaluations,
- utility and wealth improve relative to neutral allocation behavior,
- allocation behavior becomes less noisy over training.

Why it matters

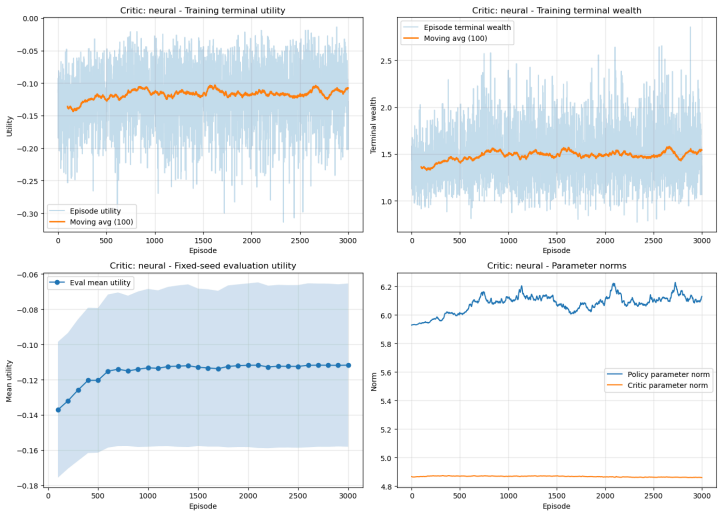
This is the first behavioral convergence test. The model must not only optimize numerically; it must converge to an economically rational response.

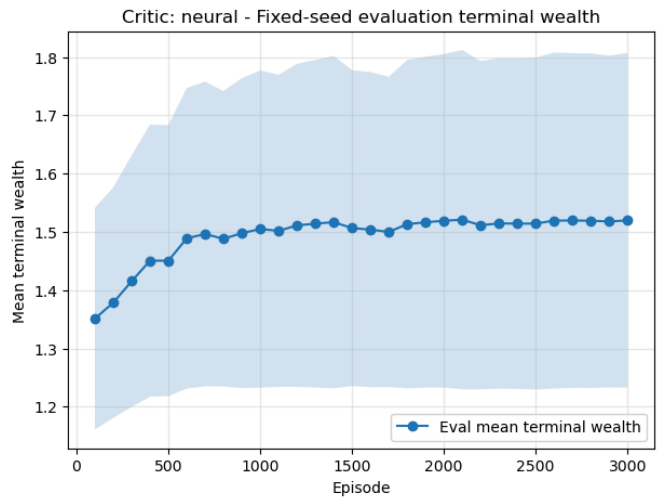
Results & interpretation

In the Single Advantage Market, the neural policy learns the expected behavior: it assigns a substantially larger weight to the advantaged risky asset while still maintaining diversification. The final mean portfolio weights [0.1747, 0.4750, 0.1659, 0.1844] show that the policy clearly prefers the correct asset, and the positive bias to risky asset 1 confirms this quantitatively. At the same time, the policy does not become overly concentrated, suggesting that it balances return-seeking with risk control.

This is reflected in performance: the final mean utility is −0.1144 and the mean terminal wealth is 1.5028. The training and fixed-seed evaluation curves both show gradual improvement, with most gains occurring in the early phase of learning and then stabilizing later. In addition, the relatively low average turnover of 0.0994 indicates that the strategy is stable and does not depend on excessive rebalancing. Overall, the results suggest that the neural actor-critic method

successfully identifies the single favorable asset and converts this structural advantage into better portfolio performance.





4.5 Experiment 3: Two-state rotation environment

Design purpose

This is the key dynamic environment. Different assets are favorable in different regimes, so the policy must learn a state-dependent allocation rule rather than a fixed average portfolio.

What convergence means here

Convergence in this setting is stronger and more demanding. It requires:

- stable training behavior,
- distinct conditional allocations across regimes,
- persistent state response,
- acceptable conditional accuracy.

Validation metrics

The most important diagnostics here are:

- **conditional accuracy**,
- **state response of asset 1**,
- **state response of asset 2**.

These metrics directly test whether the policy has learned a coherent mapping from state to action.

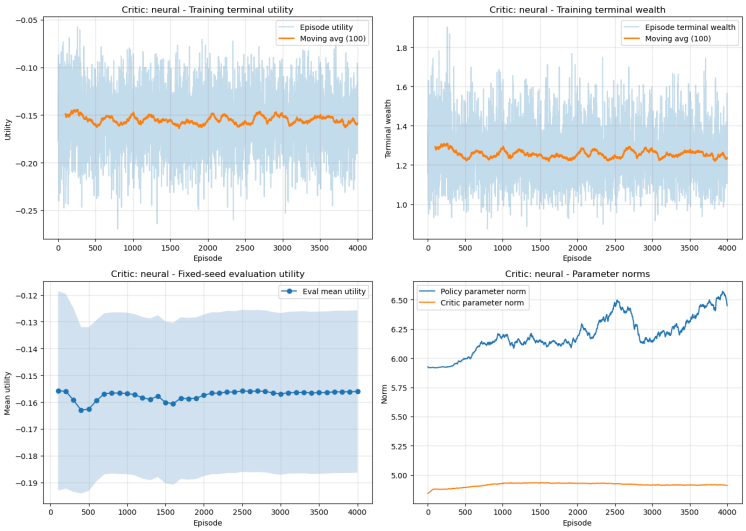
Why it matters

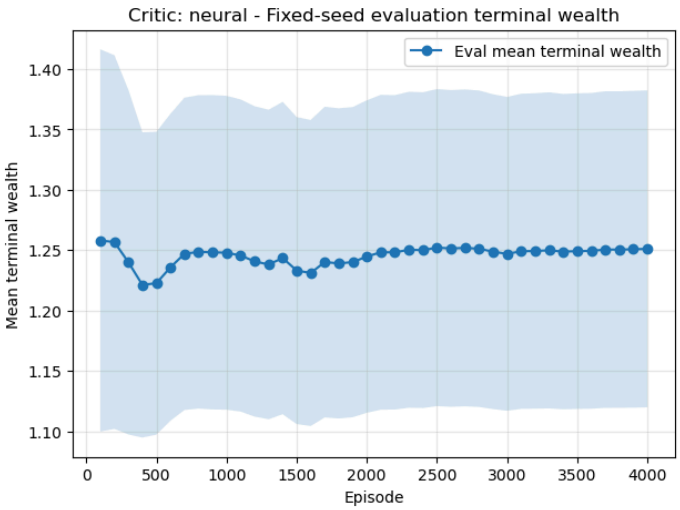
This experiment distinguishes true dynamic learning from static averaging. A policy that merely converges to a constant portfolio has not solved the dynamic allocation problem.

Results & Interpretation

In the **Two-State Rotation Market**, the policy shows clear but limited evidence of learning the regime structure. The final **mean utility** is −0.1562 and the **mean terminal wealth** is 1.2499, both lower than in the single-advantage case, which reflects the greater difficulty of a market where the favorable asset changes with the state. The average portfolio weights indicate a persistent baseline preference for the first asset, but the conditional weights show meaningful state dependence: in regime 0 the policy allocates more to one rotating asset, while in regime 1 it shifts weight toward the other.

This is confirmed by the **conditional accuracy of 0.6917**, which is substantially above random guessing, showing that the agent captures useful state information. However, the **state response magnitudes** are only around 0.05, meaning that the allocation changes across regimes are relatively small. Therefore, the learned policy reacts in the correct direction but not strongly enough to fully exploit the rotation opportunity. Overall, the experiment suggests that the neural policy learns a partially successful state-contingent allocation rule, but the regime-switching behavior remains too weak to achieve high terminal wealth.





4.6 Experiment 4: Turnover-friction scan

Design purpose

The turnover scan is designed to test not only performance robustness, but also whether policy convergence depends on the degree of trading freedom. The scan compares maximum turnover levels: [0.2, 0.1, 0.05, 0.02]

What convergence means here

If convergence is genuine, then the learned policy should respond systematically to changes in the turnover constraint. The experiment checks whether learned behavior remains coherent and whether training becomes more or less stable as turnover freedom changes.

Final results

Max Turnover	Mean Utility	Mean Wealth	Avg Turnover	Conditional Accuracy	State Response A1	State Response A2
0.20	-0.1658	1.2043	0.2000	0.5789	0.0284	0.0224
0.10	-0.1571	1.2459	0.1000	0.6767	0.0471	0.0420
0.05	-0.1530	1.2672	0.0500	0.6833	0.0562	0.0512
0.02	-0.1505	1.2812	0.0200	0.6872	0.0630	0.0580

Convergence interpretation

This experiment provides the strongest validation evidence in the project. As the turnover cap becomes tighter:

- utility improves,
- wealth increases,
- conditional accuracy rises,
- state response strengthens.

This means tighter turnover constraints did not merely reduce trading mechanically. They appear to have improved the **quality and stability of the learned policy**. In convergence terms, this suggests that looser trading freedom created noisier and less disciplined policy updates, while tighter constraints regularized the learning process.

4.7 Main conclusion on convergence validation

Taken together, the four experiments verify convergence in a layered way:

- **static benchmark** validates numerical and mechanical stability;
- **single-advantage environment** validates convergence to a basic rational allocation rule;
- **two-state environment** validates convergence to a dynamic state-dependent policy;
- **turnover scan** validates robustness and shows how constraints affect convergence quality.

This design is important because it shows that convergence is not inferred from one upward learning curve, but from a collection of consistent optimization and behavioral signals.

5. Economic Interpretation

The empirical results support several economic interpretations.

5.1 The learned policy is not just numerically optimized

The experiments indicate that the policy learned behaviors that are economically meaningful:

- it can stabilize in a stationary setting;
- it can overweight a persistently superior asset;
- it can differentiate allocations across regimes;
- it responds systematically to portfolio constraints.

This gives the results more credibility than a pure reward-based evaluation.

5.2 Turnover constraints act as regularizers

The most important economic insight comes from the turnover scan. A naive intuition would suggest that higher turnover flexibility should help in a regime-switching environment because the policy can react faster. But the results show the opposite.

When the policy is allowed to trade more aggressively, it does not become better at adaptation. Instead, it becomes less accurate, less state-responsive, and less useful. This implies that turnover constraints serve two roles:

1. **market friction**, limiting immediate reallocation;
2. **implicit regularizer**, suppressing overreaction to noisy short-term signals.

This is particularly relevant in financial RL, where excessive action flexibility can amplify instability.

5.3 More flexibility is not always better

A broader lesson of the project is that a richer action space or looser implementation constraint does not necessarily improve learning. In noisy environments, unrestricted reallocation can lead to overtrading and unstable policies. Carefully designed constraints can improve both implementability and generalization.

6. Conclusion

This project develops a modular reinforcement learning framework for dynamic asset allocation under realistic portfolio constraints. The central modeling choice is to treat portfolio allocation as a **policy-learning problem**, with policy gradient as the conceptual foundation. This choice is appropriate because it directly optimizes the allocation rule, naturally accommodates continuous portfolio actions, and aligns with long-horizon utility-based objectives.

At the same time, the project shows that pure policy gradient is not sufficient in practice. To make learning stable and meaningful, the framework introduces a sequence of optimizations: **baseline-based variance reduction, actor-critic learning, neural function approximation, simplex projection, and turnover-aware execution**. These are best understood as successive refinements of the same underlying policy-gradient logic.

A major contribution of the project is its approach to **convergence validation**. Rather than treating reward increase as sufficient evidence, the report uses four controlled environments to verify stability, behavioral correctness, dynamic state responsiveness, and robustness to implementation constraints. This experimental design makes the learning results substantially more credible.

The most important substantive finding is that **tighter turnover constraints improved performance and behavioral quality**, rather than harming them. This suggests that in reinforcement-learning-based portfolio allocation, financial constraints may serve not only as frictions, but also as effective regularizers that stabilize policy learning.

Overall, the project argues that reinforcement learning is a promising framework for constrained dynamic asset allocation, but only when optimization design, feasibility enforcement, and convergence validation are developed together.